Organización técnica del entorno

Proyecto StockMind

Sistema de inventario predictivo para pequeños almacenes de productos perecederos

Fernando Ortega | José Montoro | Samuel Gallego

Organización técnica del entorno

Hemos definido cómo vamos a organizar el entorno para desarrollar, probar y desplegar StockMind. Buscamos que sea sencillo y que lo podamos mantener entre tres personas, sin montar infraestructura que luego no sepamos llevar. Lo planteamos todo en local, con servicios gratuitos, porque el proyecto es un prototipo con datos ficticios.

1. Repositorio

- Ubicación: un repositorio privado en GitHub, creado por un miembro del equipo. Los otros dos se añaden como colaboradores.
- Estructura de carpetas: una carpeta backend (Spring Boot), una frontend (HTML, CSS, JS), una database (scripts SQL y datos de ejemplo) y una docs (documentación). En la raíz van el docker-compose.yml y el README.md.
- Ramas: main solo con versiones estables, develop para integrar y una rama por tarea (por ejemplo feature/5-1-login). Antes de pasar a develop, otro compañero revisa el cambio con un pull request.
- Normas de trabajo: commits con mensajes claros, nada de subir contraseñas (usaremos un .gitignore y un archivo de configuración de ejemplo) y no tocar main directamente.

2. Entorno de desarrollo

Cada miembro trabaja en su propio ordenador con las mismas herramientas para evitar el típico «en mi equipo funciona»:

- JDK 17, Maven e IntelliJ IDEA Community o VS Code.
- Git para el control de versiones.
- DBeaver o MySQL Workbench para revisar la base de datos.
- Postman para probar la API.
- Navegador con las herramientas de desarrollador.

Las versiones se fijan en el pom.xml y en el README, para que todos usemos las mismas.

3. Base de datos en desarrollo

Cada miembro tendrá su propia base de datos local en un contenedor Docker, definida en el docker-compose.yml. Así nadie rompe los datos de otro y todos partimos del mismo esquema. El esquema y los datos de ejemplo se guardan como scripts SQL en la carpeta database y se versionan en Git. Cuando alguien cambia una tabla, sube el script y el resto lo ejecuta.

4. Entorno de pruebas

- Pruebas automáticas: tests con JUnit en el propio proyecto, que se ejecutan con Maven.
- Entorno de pruebas completo: una réplica en contenedores Docker (aplicación y base de datos) con la configuración final, para la fase 9.2.
- Datos: solo datos ficticios en CSV, nunca datos reales.
- Pruebas manuales: en navegador de escritorio y en móvil, para comprobar el diseño responsive.

5. Sistema de despliegue

El despliegue será con Docker y Docker Compose. Se empaquetan el backend y la base de datos para que la aplicación arranque igual en cualquier ordenador con Docker instalado. Para ponerla en marcha basta con clonar el repositorio y lanzar un solo comando. La guía paso a paso irá en la carpeta docs (tarea 9.3). No contratamos hosting, porque la entrega es un prototipo que se ejecuta en local.

6. Almacenamiento de documentación

- Documentos de trabajo (informe, manual de usuario, bocetos): en una carpeta compartida de Google Drive o OneDrive, para editarlos a la vez.
- Versiones finales: una copia en PDF en la carpeta docs del repositorio, de modo que código y documentación queden juntos en la entrega.
- Nombres: un criterio común, tipo 01_Analisis_v1.docx, para no perdernos con las versiones.

7. Copias de seguridad

- Código: el repositorio de GitHub es la copia remota, y además cada miembro tiene una copia local completa al clonarlo.
- Documentación: el almacenamiento en la nube guarda el historial de versiones. Además, antes de cada entrega hacemos una copia manual en un ZIP.
- Base de datos: los scripts SQL del repositorio permiten reconstruirla desde cero. Como los datos son ficticios, no hace falta una copia periódica, pero se puede sacar un volcado con mysqldump (o pg_dump) antes de cambios grandes.

8. Acceso de los miembros y reparto de tareas

Recurso	Quién accede	Permisos
Repositorio GitHub	Los tres miembros	Colaboradores con escritura. Uno de ellos es el propietario
Rama main	Todos	Solo se actualiza mediante pull request revisado
Carpeta compartida en la nube (incluye la tabla de tareas)	Los tres miembros	Edición
Base de datos local	Cada miembro, la suya	Control total en su equipo

Las credenciales no se comparten por el chat ni se suben a Git. Cada uno usa su cuenta y su configuración local.

Para repartir el trabajo usamos la propia EDT, sin ninguna herramienta extra. Cada tarea (1.1, 2.3, 5.2…) tiene un responsable asignado en una tabla sencilla dentro de la carpeta compartida, con tres columnas: tarea, responsable y estado (pendiente, en curso o hecha). Además, nombramos las ramas de Git con el código de la tarea, así se ve en el repositorio quién hace qué.